

把团队工作沉淀为 AI 可用的知识库

把团队工作沉淀为 AI 可用的知识库

——基于 LLM Wiki 范式、三个落地技能与 KaaS 团队共享

内部分享

2026年 6 月

一、我们面对的问题

我们正在推进 AI 转型。其中一个核心诉求是：把过去所有工作的内容沉淀下来，变成一个知识库，方便大模型读取、分块处理，也方便后续应用调用。

但市面上常见的"知识库"方案（各种 RAG、向量检索中台）有几个问题：

- 搭建成本高：要向量库、要分块、要 embedding 服务、要检索调优。
- 分块会切断语义：一篇完整的制度文档被切成几十个 chunk，文档之间的关联和引用关系丢失了。
- 检索有损：向量检索是"大概相关"，不是"精确命中"，关键阈值或规则可能检索不到。

我们需要一种更轻、更准、更适合团队知识量级的方案。这就是卡帕西（Andrej Karpathy）今年提出的 LLM Wiki 范式。

二、什么是卡帕西 LLM Wiki

2026 年 4 月，Andrej Karpathy 在 GitHub 发布了一篇 gist (5900+ star)，提出了一个用 LLM 维护个人或团队知识库的范式。它不是一个产品，而是一套可复制的方法论。

核心架构：三层结构

层	目录	职责	可变性
原文层	raw/	完整原文归档，source of truth（事实的唯一来源）	只读，永不修改
提炼层	distill/	AI 直接调用的结构化知识，互相链接的 markdown	可重新编译、可改
Schema 层（规则层）	CLAUDE.md	告诉 AI 怎么组织 wiki、按什么规则维护	随团队规范演进

关键原则：distill 层的每一条规则都必须能追溯到 raw 层的某个文件。追溯不上的，写进"待验证"区，不硬塞。这就是为什么这套知识库是"可信"的。

三个核心操作

卡帕西把维护知识库的工作归纳为三个操作，每个操作都不可或缺：

操作	做什么	一句话
Ingest 摄取	读新文档 → 写摘要 → 更新实体页 → 更新索引 → 记日志	把新材料"消化"进知识库
Query 查询	对知识库提问 → 定位 → 综合回答带引用 → 好答案回填	查 wiki，而不是重读原文
Lint 健康检查	定期体检：找矛盾、过期、孤儿页、缺失链接	防止知识库随时间腐烂

raw 是 source of truth

wiki 是"持续增值的资产" (compounding artifact)

schema 让 LLM 变成一个"有纪律的wiki 维护者" (disciplined wiki maintainer) 。

为什么比 RAG 更适合我们

卡帕西原文和社区实践都指出一个关键判断：

- 在约 5-10 万 token（约 150-200 页）以内，把 wiki 直接塞进大模型上下文，完胜 RAG。
- 供应商管理知识库提炼层目前约 5300 行，远低于这个阈值。
- RAG 的代价：要向量库、要分块、分块会切断 wiki 页之间的链接语义、检索有召回率损失。
- Wiki 的好处：100% 检索可靠、零基建、能全局推理、页之间互相链接涌现。

结论：我们的量级，上 RAG 是过度设计。

三、已经做到了哪一步

供应商管理知识库已经按卡帕西三层结构落地并运行。以下是当前的真实状态：

指标	数值
提炼层 (distill/)	11 个分册, 5353 行结构化知识
原文层 (raw/)	44 个核心文档归档
健康判断层 (notes/)	8 个文件 (冲突记录、待验证、摄取日志等)
Schema 层 (CLAUDE.md)	已建立, 定义三操作规范
来源标注覆盖率	337/337 =100% (每条规则都可追溯)
健康度评分	100/100, 所有检查通过
自动化巡检脚本	wiki_health_check.py (Python 标准库, 零依赖)

四、三个落地技能

为了让团队能方便地维护知识库，把卡帕西的三个操作固化成了三个 AI 技能。同事不需要懂底层逻辑，对 AI 说一句话就能触发对应流程。

 wiki-skills-三技能包.zip
13.47 KB

 llm-wiki-pm.zip
141.98 KB

 llm-wiki-plugin.zip
307.01 KB

技能一：wiki:ingest（整理）

对应 Ingest 操作。把指定文件夹里的工作文档批量整理进知识库。

它能做什么

- 扫描一个文件夹（支持 Word/Excel/PPT/PDF/HTML/Markdown全格式）

- 自动转成 Markdown (底层用 markdown)
- 归类、编码、提炼成 wiki 页, 写入三层结构
- 更新交叉链接和索引, 记录摄取日志

完整流程 (六步)

1. 扫描文件夹, 做转换计划 (列出所有文件 + 初判类别), 交人确认
2. 归档原文到 raw/, 一字不改
3. 提炼进 distill/——只留可复用的方法论和关键参数, 细节指向 raw
4. 更新交叉链接 (向上/向下/横向三个方向)
5. 更新索引 (00_索引与导航.md)
6. 记录摄取日志 (notes/ingest日志.md)

触发方式: 对 AI 说"把这些文档整理进知识库"、"wiki 化这个文件夹"、"批量归档并提炼"。

技能二: wiki:query (读写)

对应 Query 操作。基于知识库回答问题, 而不是凭通用知识瞎编。

核心原则

- 先查后答: 永远先读 wiki 再回答
- 引用来源: 每条结论标注来自哪个 distill 段落 + 哪个 raw 文件
- 发现缺口就记: wiki 里没有的, 记进"待验证", 不硬编
- 好答案可回填: 有价值的问答可以沉淀成新页

完整流程 (五步)

1. 读索引 (00_索引与导航.md), 定位答案在哪个分册
2. 读对应 distill 分册
3. 必要时回到 raw/ 看原文上下文
4. 引用来源回答, 标注冲突 (如有)
5. 发现缺口回填, 记录待验证

触发方式: 对 AI 说"查一下 wiki 里有没有 X"、"这个规则是什么"、"知识库怎么规定的"。

技能三: wiki:health (健康)

对应 Lint 操作。体检三层结构的健康度, 防止知识库随时间腐烂。

它检查什么

检查项	目标	说明
来源覆盖率	≥95%	distill 关键段落是否都标了[来源]
raw 完整性	0 缺失	distill 引用的 raw 文件是否都存在
冲突状态	逐个记录	notes/冲突记录.md 有多少未解决
孤儿页	无	没有任何其他页链接到它的"死页"
过期声明	无	raw 已更新但 distill 没跟着改
缺失交叉引用	尽量补	明明相关却没互相链接的两个页

触发方式：对 AI 说"检查一下知识库"、"跑个体检"、"wiki 健康度怎么样"。

五、落地路径：从个人到团队

知识库不是一个人一次性建成的。我们设计的路径是"个人先行 → 人工确认 → 团队共享"，每一环都有明确的边界：

整体流程

环节	在哪里做	用什么	谁负责
1. 个人整理工作知识	本地	wiki:ingest (本地 markdown)	每个人自己
2. 确认哪些适合团队共享	本地 → 判断	人工筛选	文档作者
3. 提炼并确认准确性	本地	wiki:ingest的提炼流程	文档作者 + AI
4. 上传到团队知识库	KaaS 知识库中台	JoySpace 录入流程	文档作者
5. 团队成员查询调用	AI 对话中	kaas-kb 技能	所有同事

关键分工：本地 wiki 是"生产车间"，KaaS 是"共享仓库"

- 本地 wiki (raw/distill/notes+ CLAUDE.md)：私密、迭代快、零基建、量级小用上下文就够。这是每个人自己沉淀知识的地方。
- KaaS 知识库中台：跨人检索、权限管理、版本控制——这些本地 markdown 给不了。这是团队共享的地方。
- 两者不是二选一，而是流水线的上下游。kaas-kb 技能封装了 KaaS 的 6 个工具，同事说"查一下声纹支付的 PRD"就能自动调用。

与现有系统的关系

系统	角色	何时用
本地 wiki（三层结构）	知识的"生产与编译"	个人沉淀、提炼、迭代——在这里把原始文档变成结构化 wiki
KaaS 知识库中台	知识的"共享与检索"	跨项目查询、写 PRD、团队协作——已收录声纹/aipay/支付基础等项目
JoySpace（飞书）	知识的"发布入口"	把确认的 wiki 内容录入团队库的渠道

六、团队共享：KaaS 知识库使用指南

前面讲的是"个人如何沉淀知识"。这一章讲"如何把个人沉淀变成团队共享资产"——这就是 KaaS 知识库中台的作用。

KaaS 是什么

KaaS 是事业群级的知识库中台，把散落在各团队的 PRD、实现文档、测试用例、架构说明沉淀到一个统一的地方，通过两种方式对外提供服务：

- Web 管理台——人用，做项目维护、文档录入、召回调试。
- MCP 服务——AI Agent用，让 AI 写 PRD / 改代码 / 写用例时自动拿到历史全量上下文。

信息	内容
管理台地址	http://ai-kaas.pre-apps.jd.com
MCP 接口	https://ai-analysis-api.jd.com/mcp/ (注意必须带尾斜杠)
数据源	Git 仓库 knowledge-base.git (master 分支)
适用场景	写 PRD、改前端/后端、写测试用例、二期开发拿历史上下文
不适用	实时生产数据、线上日志、事业群以外的集团文档

核心概念：三个层级

概念	说明	例子
一级域 (L1)	事业群下的业务大块，目录第一层	payment (支付)、enterprise-finance (企业金融)
项目 (Topic)	一级域下的具体业务主题，检索最小单元	voiceprint (声纹)、aipay (京东AI付)
知识库 (KB)	项目下可独立配置召回策略的库	kb_e4bbdf31 (声纹识别库)

目前已收录的项目：rental-3c (3C租赁)、payment-basic (支付基础)、aipay (京东AI付)、voiceprint (声纹识别) 等。

仓库目录结构

知识库源是 Git 仓库，按固定结构组织：

domains/一级域/项目/└─ template/模板 (PRD 章节骨架等) └─ wiki/角色 wiki (用户心智、规范、约定) └─ docs/历史 PRD、设计文档 └─ artifacts/架构图、ADR、Runbook

注意：KaaS 的目录结构和我们本地 wiki 的 raw/distill/notes 是对应的——KaaS的 wiki/ 角色知识和我们的 distill/ 提炼层本质相同，只是一个团队共享版、一个是个人本地版。

如何录入团队知识（从个人 wiki 到 KaaS）

这是闭环的关键一步。把个人本地 wiki 里确认好的内容，录入团队 KaaS：

1. 在 KaaS 管理台「项目管理」新建项目（或找到已有项目），认真填写关键词和描述——它们直接影响检索效果
2. 把本地 distill/ 里确认准确的内容，整理成 KaaS 需要的目录结构 (template/wiki/docs/artifacts)
3. 通过「文档批量录入」从 JoySpace 目录批量入库，或直接提交 Git
4. 在「知识库管理」创建知识库并绑定到项目
5. 在「召回调试」验证文档能被正确召回

关键：不是所有个人 wiki 内容都要上传团队库。只上传"团队共享有价值、已确认准确、去除个人隐私"的部分。

AI 如何调用 KaaS (MCP 六个工具)

这是同事日常最常用的部分。配置好 MCP 后，AI 能自动检索团队知识库。同事说"查一下声纹支付的 PRD"即可，不用懂底层。

工具	用途	何时用
kb_describe	服务能力自描述	第一次接入时了解能力
kb_search	关键词检索，返回片段	快速看知识库里有没有相关内容
kb_resolve	路由决策 + 候选主题文件清单	不知道属于哪个项目时
kb_pack	写作前首选：组装模板+wiki+PRD全文	写 PRD / 改代码前，一次性拿全量上下文
kb_get_artifact	读单篇完整原文	需要某篇文档全文时
kb_get_artifacts	批量读多篇原文	需要多篇文章时

强制工作流：写 PRD / 改代码前必走

这是 KaaS 服务端明确要求的流程，违反是已知反模式：

- 1. kb_search(query="需求关键词")→ 看命中哪个项目 (topic)
- 2. kb_pack(topic, role, intent,query) → 一次性拿到模板 + 角色 wiki + 历史全文
- 3. 基于 pack 返回的template 章节顺序起草
- 4. 有缺口 → kb_get_artifacts(paths=[...]) 补齐

铁律：写 PRD 前必须先 kb_pack 拿全量，不能只用 kb_search 片段就动笔——片段会漏掉入口枚举、用户心智、历史决策。

如何接入 MCP

在项目的 .mcp.json 中加入（本项目已配置）：

```
"kaas-knowledge": {"type": "http", "url":  
  "https://ai-analysis-api.jd.com/mcp/" }
```

配置后重启 AI 会话即可。仅京东内网可用。

常见问题速查

问题	解决
MCP 连接失败 (307重定向)	URL 必须带尾斜杠 /mcp/
外网访问不了	内网服务, 需接入内网或 VPN
kb_search 召回不准	检查项目的关键词和描述是否充分; 二期 top_k 建议 10-20
文档没被收录	检查 Git 仓库目录是否有文件; 管理台点「同步」触发 git pull
需要批量录入	用「文档批量录入」页面, 从 JoySpace 目录批量入库



kaas-kb-skill.zip
9.74 KB



七、现在的重点放在哪

方案已经清晰，但执行力决定成败。建议是明确优先级：

第一优先级：先把现有文档 wiki 化

- 我们已经有跑通的知识库原型（健康度 100），证明了范式有效。现在最缺的不是工具，是内容。
 - 用 wiki:ingest 把散落在各处的现有工作文档（手册、SOP、制度、模板）批量整理进知识库。

第二优先级：用真实文档打磨技能

- 三个 wiki 技能现在是 0.1 版本的骨架——流程完整，但要在"反复处理真实文档"的过程中才能成熟。
- 用 wiki 化 5-10 份文档的过程，发现哪些步骤卡壳、哪些规则要细化，持续改进。

第三优先级：推广到团队

- 用供应商管理制度的案例，向其他业务内容示范。
- 培训使用：他们只需要对 AI 说"把这些文档整理进知识库"，AI 按wiki:ingest 流程自动跑，人只负责确认归类和审核提炼结果。

八、诚实的边界：目前还做不到什么

为了建立信任，主动说明当前的局限：

- 没有"一键批量 wiki 化"的完全自动化：目前 ingest 是半自动——AI读原文 + 按规范写 distill，但分类、编码、提炼质量的把关需要人。这是设计如此，不是缺陷。
- 健康检查还比较粗：脚本现在查来源覆盖、raw 完整性、冲突数。卡帕西说的"孤儿页检测、交叉引用密度"还没有完全实现，是下一步要补的。
- 有些冲突待业务确认：知识库里还有 14 个未解决冲突（不同文档对同一规则说法不同），需要业务侧逐个拍板。
- 技能还在 0.1 版本：三个 wiki 技能的流程完整，但都需要用真实文档喂养成熟。

九、下一步行动计划

时间	行动	产出
本周	用 wiki:ingest 批量整理 5-10 份文档	知识库内容扩充，标杆案例成型
下周	跑健康检查，修复发现的问题，打磨技能细节	健康度维持，技能升级到 0.2
两周内	示范，培训同事使用	推广方案，2-3 个新业务线启动
持续	强化健康检查脚本（孤儿页、交叉引用检测）	wiki:health 升级
持续	打通"本地 wiki → KaaS"的完整链路	端到端可演示

附录：参考资料

- 卡帕西 LLM Wiki 原文：gist.github.com/karpathy/llm-wiki
- 本知识库实际位置：项目 wiki/ 目录（含 raw/ distill/ notes/ + CLAUDE.md）
- 三个技能位置：`.agents/skills/` 下的 `wiki-ingest` / `wiki-query` / `wiki-health`
- KaaS 知识库技能：`.agents/skills/kaas-kb/`（封装 6 个 MCP 工具）
- 健康检查脚本：`wiki/scripts/wiki_health_check.py`（Python 标准库，零依赖）

如有疑问，欢迎随时交流。这份方案会随实践持续迭代。